SA-HEACS: Server-Assisted Hybrid Execution and AI-Optimized Cloud Shielding Against Automated Reverse Engineering

Kai Yi Ji

Preprint, 16 October 2026

Abstract

Automated, AI-assisted reverse-engineering pipelines now chain decompilers such as Ghidra, dynamic instrumentation such as Frida, and large language models to recover program semantics far faster than a human analyst. Classical client-side defences (obfuscation, anti-debugging, tamper-proofing) raise the cost of analysis but cannot prevent recovery in general, a limit made precise by the impossibility of virtual black-box obfuscation. Client/server code splitting avoids that limit by not shipping the protected code.

This paper revisits client/server code splitting for AI-assisted reverse engineering. A thin client shell holds no high-value intellectual property (IP) and reaches the product's most valuable logic, kept on servers the vendor controls, through attested remote calls; we call this design Server-Assisted Hybrid Execution and AI-Optimized Cloud Shielding (SA-HEACS). The split and the attested call path follow prior work. We contribute two mechanisms that make the split workable over a product's life: a learned orchestrator that decides execution locality, micro-batches requests and prefetches likely work to offset the latency and cost of remote execution; and a Dynamic Feature Promotion/Demotion Pipeline that moves each feature between cloud-only, hybrid and client-local tiers as its IP value and clone risk change.

We give a threat model, pseudocode and runnable Python for the attested call path, the orchestrator and the lifecycle policy, and a discrete-event simulation of cost and latency. In that simulation, a 5 ms batching window cuts accelerator time per request about eightfold for 6 ms of extra 95th-percentile latency, and prefetching the two most likely next operations hides the network round trip for 68% of steps at the price of 132% extra compute. Keeping the code on the server removes it from the attacker's machine, which defeats static decompilation and memory dumping of that code. It does not stop black-box extraction of functionality through the API: query-based model extraction, output-based cloning and account abuse remain, and we treat them as residual risks with rate-limiting, watermarking and anomaly-detection mitigations rather than claiming them solved.

Keywords: software protection; reverse engineering; client/server code splitting; obfuscation; remote attestation; computation offloading; model extraction; ML serving.

1 Introduction

Code that runs on hardware the attacker controls can eventually be read, so the strongest protection for an algorithm is not to ship it. Software protection has long accepted that every client-side defence is a cost multiplier, not a barrier [1], [2], [3]. Studies of professional attackers show that they defeat layered obfuscation by combining static and dynamic analysis and by focusing on the small region that holds the asset [4].

Two shifts make that cost multiplier smaller. First, analysis tools are scriptable end to end: Ghidra exposes a headless analyser and scripting API [5], frameworks such as angr automate symbolic and

dynamic analysis [6], and Frida injects JavaScript hooks into running processes to trace, patch and dump memory [7]. Second, learned models now lift binaries toward readable source and recover names and types: DIRE and DIRTY predict identifier names and types from decompiler output [8], [9], and LLM4Decompile trains language models to decompile x86 binaries end to end [10]. An automated reverse-engineering agent (the original blueprint for this design calls such a toolkit an 'REA') can therefore chain dumping, decompilation and semantic recovery with little human effort. The scarce resource for the attacker is no longer analyst time; it is access to the code.

Removing that access is an established defence. Client/server code splitting moves selected code to a trusted server so that the shipped client is incomplete; it goes back at least to DISSECT [11], was applied to program slices [12], combined with remote attestation [13], and has been measured against human attackers [14]. Offloading work from a constrained client is likewise well studied, with performance rather than secrecy as the goal [15], [16], [17].

This paper revisits client/server code splitting for AI-assisted reverse engineering. We keep an application's high-value logic on servers the vendor controls and leave the client a thin shell that handles input, local rendering and commodity features and reaches that logic through attested calls. We call this design Server-Assisted Hybrid Execution and AI-Optimized Cloud Shielding (SA-HEACS). The split and its attested call path follow prior work (Section 2); what we add is the machinery to run the split economically and to change it as a product ages.

This paper makes two contributions:

1. A learned orchestrator that decides execution locality under a hard security constraint and uses micro-batching and Markov prefetch to offset the cost and latency of remote execution, with pseudocode and runnable Python (Section 5).
2. A Dynamic Feature Promotion/Demotion Pipeline that scores each feature's IP value and clone risk and moves it between cloud-only, hybrid and client-local tiers over the product's lifecycle (Section 6).

To support them, we describe the split architecture and its attested request protocol, which follow established code-splitting and attestation designs (Section 4); give a threat model for AI-assisted reverse-engineering pipelines that states which attacks the split removes and which it does not (Sections 3 and 7); and provide an evaluation methodology and a reproducible simulation of cost and latency trade-offs, with the limits of what simulation can show (Section 8). All code is listed in Section A and runs as shipped; every program output quoted in the text is read from the output of that code.

2 Background and related work

Prior work establishes that client-side protection only delays analysis, that moving sensitive code to a trusted server is an established and empirically evaluated defence, and that a remote API leaks its function through its outputs. SA-HEACS builds on those lines; Section 2.8 states what it adds.

2.1 Obfuscation and its limits

Collberg, Thomborson and Low catalogued obfuscating transformations by potency, resilience and cost [1], and later framed obfuscation, watermarking and tamper-proofing as complementary tools [2], [18]. Barak et al. proved that virtual black-box obfuscation is impossible for general programs: some functions leak properties through any obfuscated form [19]. Indistinguishability obfuscation is constructible in theory [20] but remains far too slow for production code. Empirically, Schrittwieser et al. found that most practical obfuscations fall to modern static and dynamic analysis given enough effort [3], and Banescu et al. showed that symbolic execution defeats many transformations unless they are designed

against it [21]. Ceccato et al. observed professional penetration testers and found that they locate and attack the protected asset rather than deobfuscating the whole program [4].

2.2 Automated and learning-based reverse engineering

Scriptable platforms turn reverse engineering into a pipeline. Ghidra provides headless batch analysis and a scripting API [5]; angr packages the offensive binary-analysis techniques surveyed by Shoshitaishvili et al. [6]; Frida injects instrumentation into live processes on desktop and mobile platforms [7]. Learned models then improve the output: DIRE and DIRTY predict variable names and types for decompiled code [8], [9], and LLM4Decompile reports end-to-end decompilation of compiled binaries by fine-tuned language models [10]. These tools lower the human effort per target, which weakens defences whose value rested on analyst time.

Recent work targets LLM-based agents directly. A BAR 2026 study examines where agents fail on capture-the-flag reverse-engineering tasks, as a basis for LLM-resistant software protection [22], and a 2026 practitioner report explores how far AI-assisted reverse engineering can be hindered [23]. Both address code that the attacker holds; SA-HEACS is complementary and removes the most valuable code from the attacker's machine.

2.3 Client/server code splitting

Client/server code splitting moves part of a program to a trusted server so that the client cannot run, or be fully analysed, without it. DISSECT introduced the technique, distributing selected parts of an application to a server for security [11]. Zhang and Gupta later hid critical program slices on a trusted server so that the client binary is incomplete [12], and the bianlian work studied how to balance execution between client and server in remote tamper-resistance, with the server continuously replacing client code [24].

Code splitting has also been evaluated empirically. Viticchie et al. measured the effort needed to attack programs protected with client/server code splitting and found that attack success fell from 89% on clear code to 52% with the strongest protection [14]. Splitting raises attack cost substantially without stopping every attacker, which is consistent with the residual risks in Section 7.

2.4 Computation offloading

Offloading systems such as MAUI [15], CloneCloud [16] and Neurosurgeon [17] partition work between device and cloud to save energy or latency, and cloud gaming systems stream rendered output to thin clients [25]. These systems optimise performance; secrecy of the offloaded code is not their goal, so they freely move code to the client when that is faster. SA-HEACS reverses the priority: the security tier is a hard constraint and performance is optimised inside it.

2.5 Attestation

Remote attestation lets a server judge the state of a client before serving it. RFC 9334 defines the roles of attester, verifier and relying party [26]. Mobile platforms provide attestation services, Google Play Integrity [27] and Apple App Attest [28], and trusted execution environments such as Intel SGX add hardware-rooted quotes [29]. Attestation raises the cost of driving the API from a modified client, but it does not keep a legitimate client from observing its own inputs and outputs.

Reactive attestation is the closest prior work to the attested call path of SA-HEACS. Viticchie et al. couple remote attestation with client/server code splitting, so that a server that detects tampering reacts by no longer serving the split code to that client [13]. Our call path (Section 4) follows the same pattern of gating server-side execution on attestation, with a device key bound to platform attestation.

2.6 Model extraction through APIs

Keeping code on a server does not keep its function secret. Tramèr et al. extracted models from commercial prediction APIs through queries alone [30]; Orekondy et al. trained functionally similar 'knockoff' networks from black-box outputs [31]; Jagielski et al. studied high-fidelity extraction [32]. Defences include query-distribution monitoring such as PRADA [33] and watermarks embedded in a model's behaviour [34]. These results bound what SA-HEACS can claim (Section 7).

2.7 Serving systems and batching

The orchestrator draws on ML serving work. Clipper introduced adaptive batching for latency-bounded prediction serving [35]; Orca added iteration-level scheduling for transformer inference [36]; vLLM's PagedAttention raised batch capacity by managing key–value cache memory [37]. Resource Central predicts virtual-machine behaviour to inform allocation [38], and predictive prefetching has long been used to hide web latency [39]. Dean and Barroso describe why tail latency, not the mean, governs user experience at scale [40].

2.8 How this paper differs

SA-HEACS does not claim the client/server split, server-side execution of protected code, or attested remote calls as new; all three follow the work above. Its contributions are narrower. First, a learned orchestrator decides per request where `hybrid` and `client_ok` work runs, never moves `cloud_only` work to the client, and uses micro-batching and prefetch to offset the cost and latency that remote execution adds (Section 5). Second, the Dynamic Feature Promotion/Demotion Pipeline treats the split as something that changes over a product's lifecycle, moving each feature between cloud-only, hybrid and client-local tiers as its IP value and clone risk change (Section 6). We frame both for attackers who run AI-assisted pipelines combining Ghidra, Frida and LLMs, against which the time-based value of client-side protection shrinks while code that is not shipped stays out of reach.

3 Threat model

The attacker fully controls the client device and holds a valid account, but cannot read server memory or storage. This is the standard man-at-the-end setting [18] with a trusted server added.

Assets. (A1) Proprietary algorithms and model weights in cloud-only features. (A2) Server-side state such as cached intermediate results. (A3) The service's availability and cost envelope. (A4) Lower-value code shipped in the client shell, which is explicitly not protected beyond standard obfuscation.

Attacker capabilities. The attacker can decompile and patch the client binary, hook any function with dynamic instrumentation, read and modify process memory, intercept traffic after TLS termination on their own device, and script all of this with an AI agent that plans, runs tools and summarises findings. They can create many accounts, subject to the vendor's sign-up controls, and send any sequence of valid queries.

Trust assumptions. The server fleet, its hypervisor and the build pipeline are trusted; insiders and cloud-provider compromise are out of scope. Platform attestation services [27], [28] are assumed sound, though we treat a bypassed attestation as a degraded mode rather than a total failure. Side channels on shared server hardware are out of scope but discussed in Section 9.

Attacker goals. Table 1 lists the goals we consider and whether the design addresses them.

Table 1: Attacker goals and their status under SA-HEACS.

Goal	Example	In scope
G1 Code recovery	Recover source or weights of a cloud-only feature	Yes
G2 Functional cloning	Train or build a substitute from input–output pairs	Yes, as residual risk
G3 Free-riding	Use a cloned or patched client to call the API without paying	Yes
G4 Shell cloning	Copy the client UI and commodity features	Accepted
G5 Denial of service	Exhaust GPU capacity with costly requests	Partly
G6 Server compromise	Exploit the RPC surface to read server memory	Out of scope; standard hardening

4 SA-HEACS architecture

SA-HEACS has three components joined by one attested request path: the client shell, the orchestrator and the secure clusters (Figure 1).

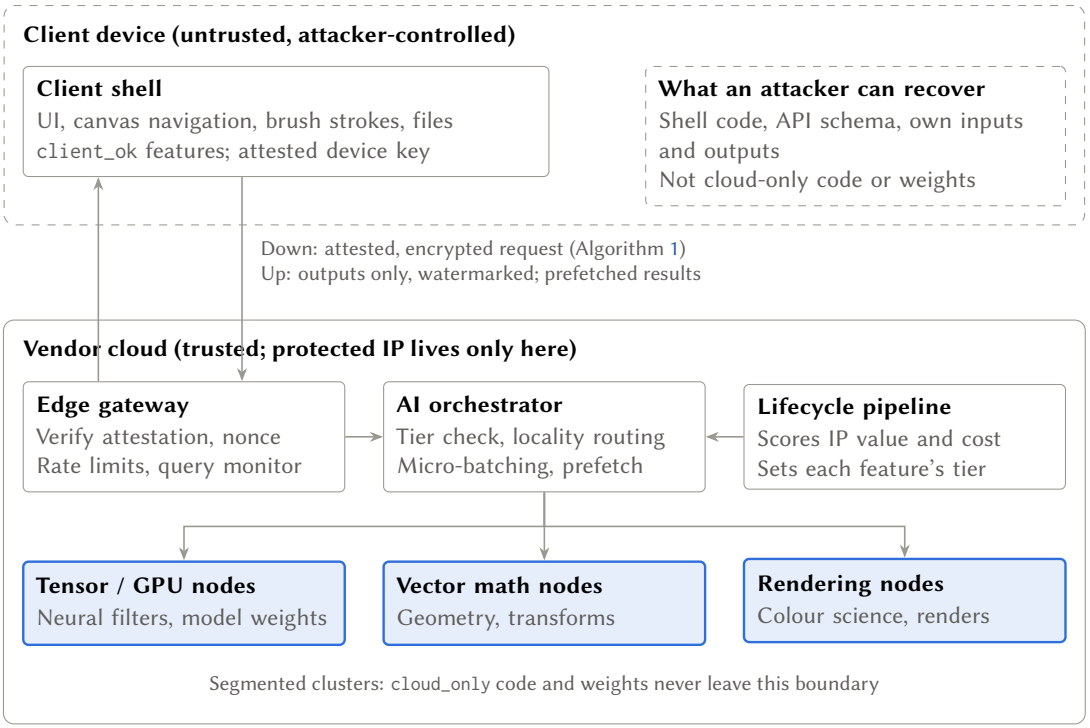


Figure 1: SA-HEACS architecture: client shell, attested RPC, orchestrator and segmented clusters. Everything inside the dashed client boundary is assumed readable by the attacker; the blue clusters hold the protected IP and return only outputs.

4.1 Client shell (the untrusted boundary)

The shell handles input, canvas navigation, local brush strokes, file handling, UI rendering and any feature in the `client_ok` tier. It holds no cloud-only code or weights. Its only secrets are a device identity bound to platform attestation and short-lived session keys. We assume it will be fully decompiled and cloned; the design goal is that a perfect clone gains a UI and commodity features, not the protected capabilities.

4.2 Secure clusters (the protected core)

Cloud-only features run as segmented microservices grouped by hardware need: tensor and GPU inference, vector and geometry maths, and rendering or colour-science pipelines. Each feature runs under its own service identity with least-privilege access to its weights. Clients never receive code, weights or intermediate tensors that would let a substitute be rebuilt more cheaply than by black-box queries; responses carry only the final output at the resolution the product needs.

4.3 Orchestrator (the control plane)

The orchestrator sits between the edge and the clusters. It enforces each feature's tier, routes `hybrid` and `client_ok` work to the cheaper side, groups compatible requests into micro-batches, and prefetches likely next operations. Section 5 details it.

4.4 Request protocol

Every remote call carries an attestation-bound signature, a fresh nonce and an encrypted body (Algorithm 1). The client derives a per-request key from an ephemeral X25519 exchange [41], [42] with the server's static key, expanded with HKDF [43] using the nonce as salt, and encrypts with AES-GCM [44]. The ephemeral public key, nonce and timestamp are signed with an Ed25519 device key [45], [46] that, in production, is bound to a platform attestation verdict, and the same bytes are passed to AES-GCM as associated data. The server rejects stale or replayed requests and unattested devices before any protected code runs. Request and response use disjoint 12-byte GCM nonces under the per-request key.

Algorithm 1 Attested invocation of a cloud-only feature

Client:
1: $(e, E) \leftarrow$ X25519.Generate()
2: $n \leftarrow$ Random(16); $t \leftarrow$ Now()
3: $\sigma \leftarrow$ Sign(K_{device}, $E \parallel n \parallel t$) ▷ K_{device} bound to platform attestation
4: $k \leftarrow$ HKDF(ECDH(e, S_{pub}), salt $= n$)
5: $c \leftarrow$ AES-GCM.Enc(k, $n[0:12]$, {feature, args}, aad $= E \parallel n \parallel t$)
6: **send** (device_id, E, n, t, σ, c)
Server:
7: **if** $|\text{Now}() - t| > \Delta$ **or** $n \in$ SeenNonces **then reject**
8: **if not** Verify(AttestedKey[device_id], σ) **then reject**
9: **if not** RateLimiter.Allow(account, feature) **then reject** ▷ Section 7
10: $k \leftarrow$ HKDF(ECDH(S_{priv}, E), salt $= n$)
11: req $\leftarrow$ AES-GCM.Dec(k, $n[0:12]$, c, aad)
12: $y \leftarrow$ Orchestrator.Submit(req.feature, req.args) ▷ Section 5
13: **return** AES-GCM.Enc(k, $n[12:16] \parallel 0^8$, Watermark(y), aad)

The runnable version, `attested_rpc.py` (Section A.1), implements lines 1–8 and 10–13, without the rate limiter and watermark, using the Python `cryptography` package [47] and FastAPI [48]. Running it shows a valid call returning the filtered result and a cloned client with an unattested key rejected with HTTP 401. In production, line 8 would consume a Play Integrity or App Attest verdict [27], [28] instead of a locally registered Ed25519 key, and the nonce cache would be bounded by the skew window Δ.

5 The AI orchestrator

The orchestrator's job is to make remote execution cheap and fast enough that keeping IP server-side is commercially viable. It does three things, and the security tier constrains all of them: no optimisation may move a `cloud_only` feature to the client.

5.1 Locality routing

For `hybrid` and `client_ok` features, a classifier trained on telemetry predicts whether remote execution will finish sooner than local execution, from payload size, estimated work, measured round-trip time, device capability and battery level. This follows the partitioning logic of MAUI and Neurosurgeon [15], [17], with the tier check placed first (Algorithm 2).

Algorithm 2 ROUTE(r)

1: **if** r.tier = `cloud_only` **then return** CLOUD ▷ hard security constraint
2: **if** r.tier = `client_ok` **and** r.rtt $>$ $\mathrm{RTT}_{\max}$ **then return** LOCAL
3: $p \leftarrow$ MODEL.P(remote faster | payload, work, rtt, device, battery)
4: **return** CLOUD **if** $p \geq 0.5$ **else** LOCAL

`orchestrator.py` (Section A.2) trains a scikit-learn `GradientBoostingClassifier` [49] on 3,000 synthetic telemetry records and reaches 0.956 accuracy on 1,000 held-out records. The labels come from a simple latency model, so this number shows that the pipeline works, not that a real router would be this accurate.

5.2 Micro-batching

Accelerators amortise fixed per-call cost (kernel launch, weight reads) across a batch. The orchestrator holds requests for the same feature for up to a window of W milliseconds or until B requests arrive, then runs one batched call (Algorithm 3), as Clipper does for model serving [35]. Requests from different tenants share a batch, but each row's output is returned only to its sender, and no state persists across rows.

Algorithm 3 MICROBATCHER.SUBMIT(feature f, input x)

1: $Q[f]$.APPEND((x, future))
2: **if** $|Q[f]| \geq B$ **then** FLUSH(f)
3: **else if** no timer for f **then** start timer(W, FLUSH(f))
4: **return await** future
procedure FLUSH(f)
5: items $\leftarrow Q[f]$; $Q[f] \leftarrow [\,]$
6: $Y \leftarrow \mathrm{KERNEL}_f$(STACK($x$ for $x \in$ items)) ▷ one accelerator call
7: **for** each (item, y) **do** item.future.Resolve(y)

The `asyncio` implementation in Section A.2 serves a burst of 200 requests in 4 batched calls (wall-clock time depends on the machine). Section 8 measures the cost and latency trade-off of W.

5.3 Predictive prefetch

A first-order Markov model over each user's feature sequence predicts the next operation (Algorithm 4). When the user invokes feature a, the orchestrator speculatively computes the top-k features b with $P(b \mid a)$ above a threshold and pushes the results to the client cache. A hit replaces a network round trip with a local read; a miss wastes compute. With a toy session of ten steps, the model in Section A.2 learns that after `denoise` the user goes to `upscale` or `colorize`.

Algorithm 4 ONINVOKE(user u, feature a)

1: $M[u.\text{prev}][a]$ += 1; $u.\text{prev} \leftarrow a$
2: **for** $b \in$ TOPK($M[a]$) with $M[a][b] / \sum M[a] \geq p_{\min}$ **do**
3: **if** BUDGET.ALLOW(u, b) **then** schedule speculative RUN(b, u.state) at low priority

Prefetched results are outputs, not code, so prefetch does not weaken the zero-footprint rule. It does increase the volume of outputs a client receives, which matters for extraction risk (Section 7).

6 Dynamic feature promotion and demotion pipeline

A feature's tier is a business decision that changes over time: new IP starts cloud-only, and it moves toward the client only when its secrecy is worth less than the cost and latency of serving it remotely. Moves toward the client are one-way in practice, because shipped code must be assumed public, so the pipeline makes demotion (back to the cloud) automatic and gates promotion to the client on a human reviewer.

6.1 Tiers

- **`cloud_only` (incubation).** Code and weights exist only on servers. New or high-value features start here.
- **`hybrid`.** The server stays authoritative, but an obfuscated or reduced-quality client fallback is allowed for offline use or poor networks.
- **`client_ok` (commoditised).** The feature ships in the client package, protected only by standard obfuscation [1], and runs locally to save server cost and latency.

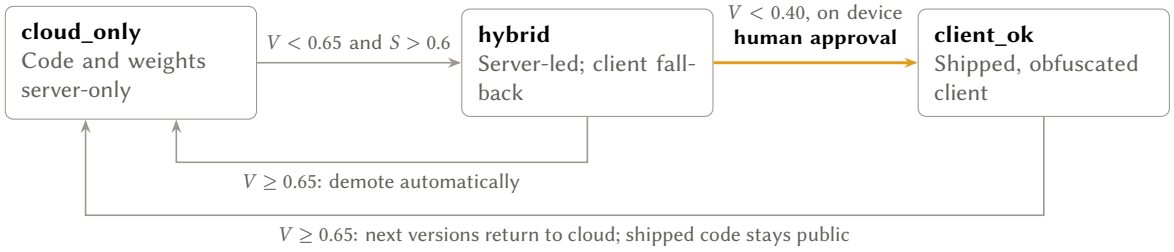


Figure 2: Feature tier state machine with the thresholds of Section 6.3 (V = IP value, S = serving pressure; every promotion first waits out a 30-day dwell time). Only the move to `client_ok` needs a person to approve it, because shipped code cannot be recalled; every move back toward the cloud is automatic.

6.2 Scoring

The pipeline computes two scores per feature from product and telemetry signals. IP value V combines novelty N, share of premium revenue R, competitor parity C (how closely public competitors match the output) and whether the method is published, $P \in \{0, 1\}$:

$$V = 0.35\,N + 0.35\,R + 0.20\,(1 - C) + 0.10\,(1 - P). \tag{1}$$

Serving pressure S compares cost per 1,000 invocations c and 95th-percentile latency ℓ to budgets $c_{\max}$ and $\ell_{\max}$:

$$S = \min\left(1,\ \tfrac{1}{2}\,\frac{c}{c_{\max}} + \tfrac{1}{2}\,\frac{\ell}{\ell_{\max}}\right). \tag{2}$$

The weights are illustrative; an organisation would set them with its product and legal teams. Clone risk enters through C and P: once competitors reach parity or the method is published, secrecy protects little.

6.3 Transition policy

The policy uses a hysteresis band ($V_{\text{hi}} = 0.65$, $V_{\text{lo}} = 0.40$) and a minimum dwell time of 30 days so that noisy signals do not make a feature oscillate between tiers (Algorithm 5).

Algorithm 5 DECIDE(feature f, signals s)

$V \leftarrow \text{IPVALUE}(s)$; $S \leftarrow \text{SERVINGPRESSURE}(s)$ ▷ Equations (1) and (2)
if $V \geq V_{\text{hi}}$ **and** f.tier $\neq$ `cloud_only` **then return** `cloud_only` ▷ demote now; reversible
if f.days_in_tier $<$ MIN_DWELL **then return** f.tier
if f.tier $=$ `cloud_only` **and** $V < V_{\text{hi}}$ **and** $S > 0.6$ **then return** `hybrid`
if f.tier $=$ `hybrid` **and** $V < V_{\text{lo}}$ **and** s.on_device_feasible **then**
    return `client_ok` pending $\text{APPROVE}(f)$ ▷ irreversible: human gate
return f.tier

`lifecycle.py` (Section A.3) runs one simulated year for a feature whose novelty and revenue share decay while competitor parity rises and the method is published on day 201. The feature moves from `cloud_only` to `hybrid` on day 97 ($V = 0.648$, $S = 0.908$) and to `client_ok` on day 201 ($V = 0.36$), the day publication removes its remaining secrecy value.

6.4 Automated packaging

A promotion to `client_ok` triggers a build step that compiles the feature for client targets, applies the shell's standard obfuscation profile, and adds it to the next client release behind a feature flag. Promotion should never ship the server's exact weights or code if a distilled or reduced version is enough, since the server version may later become valuable again or share components with newer cloud-only features.

7 Security analysis

SA-HEACS removes code recovery (G1) for cloud-only features, because the code is not on the device; it reduces but does not remove functional cloning (G2) and free-riding (G3). The original blueprint rated every row 'fully mitigated'; Table 2 separates what the architecture eliminates from what it only makes harder.

7.1 What is eliminated

Against G1, the argument is simple and does not depend on obfuscation strength: an analyst cannot recover an artefact that never reaches their machine. Barak et al.'s impossibility result [19] limits what any shipped program can hide; SA-HEACS avoids that limit by not shipping the program. This holds only if the server's responses do not contain the code or weights in another form, so responses must not include debugging traces, intermediate tensors or model files.

7.2 What remains: extraction through the API

A server-side function is still an oracle. Tramèr et al. [30] and Orekondy et al. [31] show that query access alone can produce a useful substitute model, and an AI agent can automate the query campaign as easily as it automates decompilation. SA-HEACS therefore treats G2 as a residual risk with layered controls:

- **Per-account and per-device quotas** tied to attested identities, so that query volume costs real accounts and devices.

Table 2: Reverse-engineering threats under client-only protection and under SA-HEACS.

Threat	Client-only protection	SA-HEACS, cloud-only features	Why
Static decompilation (Ghidra, IDA)	Delayed by obfuscation [1], [3]	Eliminated for the protected code	Code is not in the client package
Memory dumping and hooking (Frida)	Delayed by anti-debug and integrity checks	Eliminated for the protected code; client inputs and outputs remain visible	Computation runs server-side
AI-assisted decompilation and summarisation	Increasingly effective [9], [10]	Recovers only the shell and the API schema	Nothing protected to lift
Black-box functional cloning via the API	Possible	Remains possible; slowed by rate limits, monitoring, output shaping	Outputs leak function [30], [31], [32]
Patched or cloned client driving the API	Possible	Raised cost: attestation, per-account quotas	Attestation can be bypassed on rooted or emulated devices
Shell and UI cloning	Possible	Accepted	Clone has no protected capability
Cost exhaustion (DoS on GPUs)	Not applicable	New risk	Remote compute is costly per request

- **Query-distribution monitoring** in the style of PRADA [33], flagging accounts whose inputs look synthetic or adversarially spaced.
- **Output shaping**: return the product-level result (for example an 8-bit image) rather than logits, confidences or high-precision intermediates that speed up extraction [30].
- **Watermarking** of model behaviour or outputs [34], so that a stolen substitute can be identified even if it cannot be prevented.

These controls raise the price of a clone; none makes it impossible. The honest claim is that SA-HEACS converts a white-box attack into a black-box one and makes the black-box attack measurable and rate-limited.

7.3 Free-riding and attestation

Attestation [26], [27], [28] lets the server refuse requests from modified clients, but attackers can bypass it on compromised devices, and a legitimate unmodified client can still be scripted through accessibility or UI automation. Attestation is therefore one signal for the rate limiter and anomaly detector, not a gate that is assumed unbreakable.

7.4 New attack surface

Moving IP to the server creates risks that client-only software does not have: cost-exhaustion attacks against GPU capacity, injection or deserialisation bugs in the RPC layer, and cross-tenant leakage if batching or caching mixes state between users. Micro-batching must keep per-row isolation, and prefetch caches must be keyed per user.

8 Evaluation methodology and simulated results

In a simulation with stated assumptions, micro-batching makes server-side execution about eight times cheaper at a cost of a few milliseconds, and Markov prefetch hides most network latency but multiplies compute. These numbers come from a model, not a deployment; Section 8.3 lists what a real evaluation must measure.

8.1 Setup

`simulate.py` (Section A.4) uses SimPy [50] for a discrete-event model of one protected feature. Requests arrive as a Poisson process at 2,000 per second. A batched kernel call on b items costs $4.0 + 0.15\,b$ ms, with at most 64 items per batch; the batching timer starts when the first request enters an empty queue. The GPU pool is sized from the expected batch size so that utilisation stays at or below 70%. Each configuration simulates 20 s (about 40,000 requests), discarding the first second. For the prefetch experiment, sessions of 20 steps are walks on a skewed first-order Markov chain over 12 features; a model fitted on the first 1,000 of 3,000 sessions is evaluated on the remaining 2,000. A miss costs the network round trip (40 ms) plus one unbatched kernel call (4.15 ms); a prefetched local hit costs 5 ms. Speculative runs that are never used count as extra compute.

8.2 Results

Batching window. Figure 3 plots GPU time per 1,000 requests against server-side 95th-percentile (p95) latency for windows from 0 to 32 ms, and Table 3 gives the full results. Without batching, each request pays the full fixed cost, so 1,000 requests take 4,150 GPU-ms on a pool of 12 GPUs. A 5 ms window gathers 11 requests per batch on average and cuts this to 513 GPU-ms on 2 GPUs, while server p95 rises from 5.0 to 10.9 ms. Beyond 20 ms the fixed cost is already spread over about 40 requests, so cost falls only from 247 to 215 GPU-ms while p95 rises from 29.8 to 42.7 ms; at 32 ms most batches reach the 64-item cap.

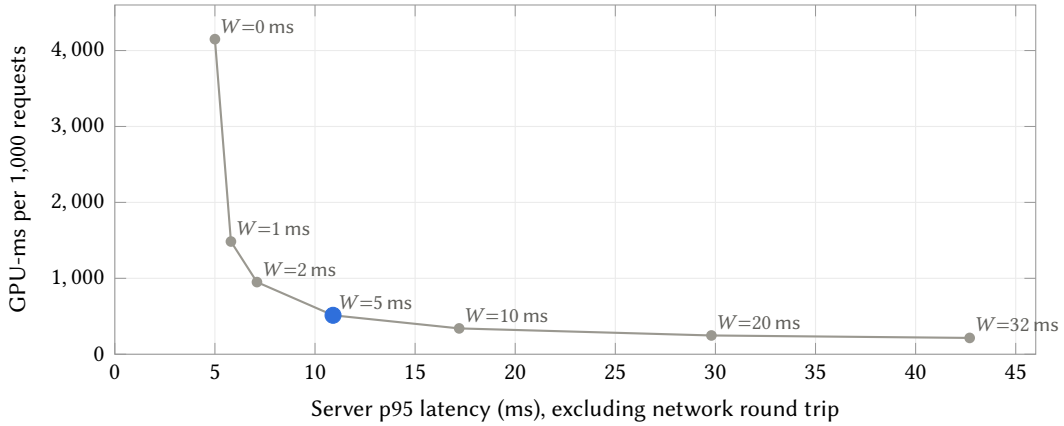


Figure 3: A 5 ms batching window (blue) cuts GPU time per request about eightfold for 6 ms more p95 latency; gains flatten past 20 ms. `simulate.py`, SimPy model, 2,000 req/s, assumed kernel costs (not measured). $W = 0$ is no batching.

The knee of this curve depends on traffic: a feature with 20 requests per second will rarely fill a batch within 5 ms, so low-traffic features gain far less. This supports the lifecycle policy in Section 6: features that cannot batch well carry higher serving pressure S and become candidates for promotion sooner.

Table 3: Batching-window results from `simulate.py` (server-side latency only).

W (ms)	GPUs	Mean batch	GPU-ms / 1k req	Util.	p50 (ms)	p95 (ms)	p99 (ms)
0	12	1.0	4, 150.0	0.69	4.1	5.0	6.0
1	5	3.0	1, 484.6	0.60	5.3	5.8	6.0
2	3	5.0	950.3	0.64	6.1	7.1	7.4
5	2	11.0	513.0	0.51	8.6	10.9	11.4
10	1	21.0	340.3	0.68	12.6	17.2	18.0
20	1	41.1	247.3	0.50	20.5	29.8	31.0
32	1	61.3	215.0	0.43	28.3	42.7	44.7

Prefetch depth. Figure 4 varies the number of speculated next features k. With $k = 2$, 68% of next steps are already in the client cache and mean perceived latency falls from 44.1 to 17.4 ms, but speculative work adds 132% to demand compute. The batching savings in Figure 3 are large enough to pay for $k = 1$ or 2; $k = 4$ adds 309% for a further 8.7 ms saving.

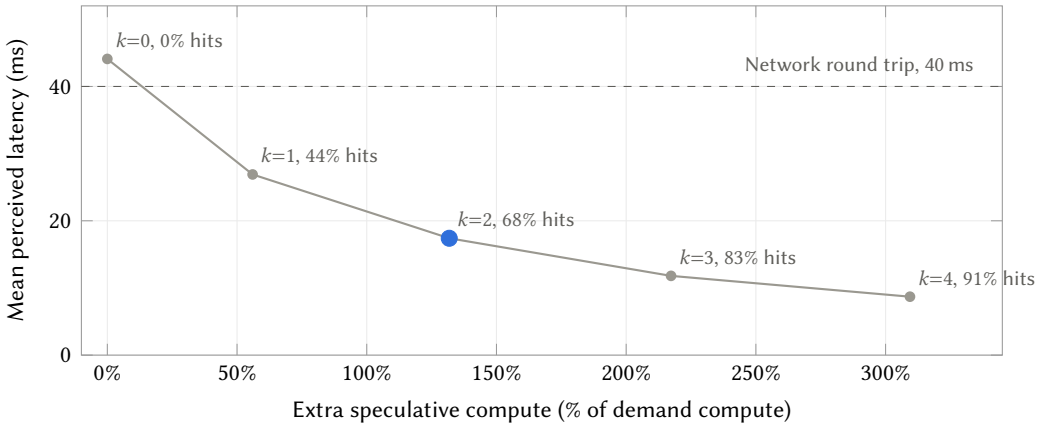


Figure 4: Prefetching the two most likely next steps (blue) cuts mean wait from 44.1 to 17.4 ms for 132% extra speculative compute. `simulate.py`, 3,000 synthetic sessions over 12 features, first third used for training.

8.3 What a real evaluation must measure

The simulation fixes kernel costs, traffic and user behaviour by assumption. A deployment study should report: (1) per-feature p50, p95 and p99 end-to-end latency across real networks, since tail latency dominates experience [40]; (2) GPU cost per 1,000 invocations with real traffic mixes and multi-tenant batching; (3) prefetch hit rate and wasted compute from real session logs; (4) a red-team exercise in which an AI-assisted reverse-engineering agent targets the client shell, reporting what it recovers; and (5) an extraction study measuring how many queries a substitute model needs to reach a given fidelity, with and without the controls of Section 7 [30], [31], [33].

9 Limitations and discussion

SA-HEACS trades a code-secrecy problem for an operations problem, and several limits remain.

- **Black-box extraction.** The architecture cannot stop a patient attacker from cloning a function through its outputs (Section 7). Features whose value is easy to learn from input–output pairs gain less from server-side execution than features with large hidden state or data.
- **Latency floor.** Interactive operations faster than about one network round trip cannot move to

the server without prediction. Prefetch helps only when user behaviour is predictable, and the 40 ms round trip assumed here is optimistic for mobile networks.

- **Offline use.** Cloud-only features fail without connectivity. The original blueprint's answer, locking or queueing premium features offline, is a product cost that users will notice.
- **Cost.** Remote execution moves compute from users' devices to the vendor's bill. Batching helps only for features with enough concurrent traffic (Section 8).
- **Privacy and compliance.** User content now leaves the device. Regulated data, data-residency rules and user trust may forbid this for some customers, and any server-side cache becomes a breach target.
- **Shared-hardware side channels.** Multi-tenant GPUs and CPUs can leak information across tenants. We scope this out, but a production system needs isolation between batches of different customers where the threat warrants it.
- **Simulation only.** Section 8 uses assumed costs and synthetic sessions. The numbers illustrate trade-offs, not achievable performance.
- **Learned control plane.** The router and prefetcher are themselves models that can drift or be gamed, for example by a user who crafts sequences to trigger costly speculation. The tier constraint is enforced in code, not learned, so a misbehaving model can waste money but cannot leak a cloud-only feature.

The lifecycle pipeline also changes how teams think about protection. Instead of asking whether a feature can be protected on the client, they ask how long its secrecy is worth the serving cost. That question has a business answer that changes over time, which is what the promotion and demotion policy encodes.

10 Conclusion

Client-side protection cannot win against an attacker who owns the machine, and AI-assisted tooling shortens the time it takes to lose. SA-HEACS accepts this and keeps the code that matters off the machine: a zero-IP client shell, attested and encrypted calls to segmented server clusters, and an orchestrator that uses locality routing, micro-batching and prefetch to keep remote execution affordable. A promotion and demotion pipeline moves each feature to the client once its secrecy no longer pays for its serving cost.

The design eliminates static and dynamic recovery of cloud-only code. It does not eliminate functional cloning through the API, which becomes the main residual risk and calls for rate limiting, query monitoring, output shaping and watermarking. Future work should evaluate the design on a real application with production traffic, run an AI-assisted red team against the client shell, and measure extraction cost against the deployed controls.

Code availability

All programs are in `paper/code/` and listed in Section A. They run with Python 3.11 or later after `pip install -r requirements.txt`; `run_all.py` runs every program, saves its output to `code/outputs/`, and regenerates the CSV files behind Figures 3 and 4.

References

[1] C. Collberg, C. Thomborson and D. Low, 'A taxonomy of obfuscating transformations,' Department of Computer Science, The University of Auckland, Technical Report 148, Jul. 1997.

[2] C. S. Collberg and C. Thomborson, 'Watermarking, tamper-proofing, and obfuscation: Tools for software protection,' *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 735–746, 2002. DOI: 10.1109/TSE.2002.1027797

[3] S. Schrittwieser, S. Katzenbeisser, J. Kinder, G. Merzdovnik and E. Weippl, 'Protecting software through obfuscation: Can it keep pace with progress in code analysis?' *ACM Computing Surveys*, vol. 49, no. 1, 4:1–4:37, 2016. DOI: 10.1145/2886012

[4] M. Ceccato, P. Tonella, C. Basile, B. Coppens, B. De Sutter, P. Falcarin and M. Torchiano, 'How professional hackers understand protected code while performing attack tasks,' in *Proceedings of the 25th IEEE/ACM International Conference on Program Comprehension (ICPC)*, IEEE, 2017, pp. 154–164. DOI: 10.1109/ICPC.2017.2

[5] National Security Agency, *Ghidra: Software reverse engineering framework*, Software, 2019. Accessed: 2026-10-16. [Online]. Available: https://github.com/NationalSecurityAgency/ghidra

[6] Y. Shoshitaishvili, R. Wang, C. Salls et al., 'SoK: (state of) the art of war: Offensive techniques in binary analysis,' in *Proceedings of the 2016 IEEE Symposium on Security and Privacy (S&P)*, IEEE, 2016, pp. 138–157. DOI: 10.1109/SP.2016.17

[7] O. A. V. Ravnås, *Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers*, Software and documentation. Accessed: 2026-10-16. [Online]. Available: https://frida.re/docs/home/

[8] J. Lacomis, P. Yin, E. J. Schwartz, M. Allamanis, C. Le Goues, G. Neubig and B. Vasilescu, 'DIRE: A neural approach to decompiled identifier naming,' in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2019, pp. 628–639. DOI: 10.1109/ASE.2019.00064

[9] Q. Chen, J. Lacomis, E. J. Schwartz, C. Le Goues, G. Neubig and B. Vasilescu, 'Augmenting decompiler output with learned variable names and types,' in *Proceedings of the 31st USENIX Security Symposium*, USENIX Association, 2022. arXiv: 2108.06363. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/chen-qibin

[10] H. Tan, Q. Luo, J. Li and Y. Zhang, 'LLM4Decompile: Decompiling binary code with large language models,' in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2024, pp. 3473–3487. DOI: 10.18653/v1/2024.emnlp-main.203

[11] E. Valdez and M. Yung, 'DISSECT: Distribution for SECurity tool,' in *Information Security – ISC 2001*, ser. Lecture Notes in Computer Science, vol. 2200, Springer, 2001, pp. 125–143.

[12] X. Zhang and R. Gupta, 'Hiding program slices for software security,' in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, IEEE, 2003, pp. 325–336. DOI: 10.1109/CGO.2003.1191556

[13] A. Viticchie, C. Basile, A. Avancini, M. Ceccato, B. Abrath and B. Coppens, 'Reactive attestation: Automatic detection and reaction to software tampering attacks,' in *Proceedings of the 2016 ACM Workshop on Software PROtection (SPRO '16)*, ACM, 2016. DOI: 10.1145/2995306.2995315

[14] A. Viticchie, L. Regano, C. Basile, M. Torchiano, M. Ceccato and P. Tonella, 'Empirical assessment of the effort needed to attack programs protected with client/server code splitting,' *Empirical Software Engineering*,

[15] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra and P. Bahl, 'MAUI: Making smartphones last longer with code offload,' in *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, ACM, 2010, pp. 49–62. DOI: 10.1145/1814433.1814441

[16] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik and A. Patti, 'CloneCloud: Elastic execution between mobile device and cloud,' in *Proceedings of the Sixth European Conference on Computer Systems (EuroSys)*, ACM, 2011, pp. 301–314. DOI: 10.1145/1966445.1966473

[17] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars and L. Tang, 'Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,' in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2017, pp. 615–629. DOI: 10.1145/3037697.3037698

[18] C. Collberg and J. Nagra, *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection*. Upper Saddle River, NJ: Addison-Wesley, 2009, ISBN: 978-0-321-54925-9.

[19] B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. Vadhan and K. Yang, 'On the (im)possibility of obfuscating programs,' *Journal of the ACM*, vol. 59, no. 2, 6:1–6:48, 2012, Preliminary version in CRYPTO 2001. DOI: 10.1145/2160158.2160159

[20] S. Garg, C. Gentry, S. Halevi, M. Raykova, A. Sahai and B. Waters, 'Candidate indistinguishability obfuscation and functional encryption for all circuits,' in *Proceedings of the 54th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2013, pp. 40–49. DOI: 10.1109/FOCS.2013.13

[21] S. Banescu, C. Collberg, V. Ganesh, Z. Newsham and A. Pretschner, 'Code obfuscation against symbolic execution attacks,' in *Proceedings of the 32nd Annual Computer Security Applications Conference (ACSAC)*, ACM, 2016, pp. 189–200. DOI: 10.1145/2991079.2991114

[22] 'Towards LLM-resistant software protection: Agent failure patterns in CTF reverse engineering,' in *Workshop on Binary Analysis Research (BAR)*, 2026. DOI: 10.14722/bar.2026.23058

[23] Quarkslab. 'Defeating AI-assisted reverse engineering (or at least trying to). '[Online]. Available: https://blog.quarkslab.com/defeating-ai-assisted-reverse-engineering-or-at-least-trying-to.html

[24] 'bianlian: Remote tamper-resistance with continuous replacement,' University of Arizona, Technical Report TR08-03, 2008.

[25] C.-Y. Huang, C.-H. Hsu, Y.-C. Chang and K.-T. Chen, 'GamingAnywhere: An open cloud gaming system,' in *Proceedings of the 4th ACM Multimedia Systems Conference (MMSys)*, ACM, 2013, pp. 36–47. DOI: 10.1145/2483977.2483981

[26] H. Birkholz, D. Thaler, M. Richardson, N. Smith and W. Pan, 'Remote ATtestation procedureS (RATS) Architecture,' Internet Engineering Task Force, RFC 9334, Jan. 2023. DOI: 10.17487/RFC9334

[27] Google, *Play Integrity API*, Android Developers documentation. Accessed: 2026-10-16. [Online]. Available: https://developer.android.com/google/play/integrity

[28] Apple Inc., *Establishing your app's integrity*, Apple Developer documentation, DeviceCheck framework. Accessed: 2026-10-16. [Online]. Available: https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity

[29] V. Costan and S. Devadas, *Intel SGX explained*, Cryptology ePrint Archive, Paper 2016/086, 2016. [Online]. Available: https://eprint.iacr.org/2016/086

[30] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter and T. Ristenpart, 'Stealing machine learning models via prediction APIs,' in *Proceedings of the 25th USENIX Security Symposium*, USENIX Association, 2016, pp. 601–618. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/tramer

[31] T. Orekondy, B. Schiele and M. Fritz, 'Knockoff nets: Stealing functionality of black-box models,' in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2019, pp. 4949–4958. DOI: 10.1109/CVPR.2019.00509

[32] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin and N. Papernot, 'High accuracy and high fidelity extraction of neural networks,' in *Proceedings of the 29th USENIX Security Symposium*, USENIX Association, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/jagielski

[33] M. Juuti, S. Szyller, S. Marchal and N. Asokan, 'PRADA: Protecting against DNN model stealing attacks,' in *Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, IEEE, 2019, pp. 512–527. DOI: 10.1109/EuroSP.2019.00044

[34] Y. Adi, C. Baum, M. Cisse, B. Pinkas and J. Keshet, 'Turning your weakness into a strength: Watermarking deep neural networks by backdooring,' in *Proceedings of the 27th USENIX Security Symposium*, USENIX Association, 2018. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/adi

[35] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez and I. Stoica, 'Clipper: A low-latency online prediction serving system,' in *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2017. [Online]. Available: https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw

[36] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim and B.-G. Chun, 'Orca: A distributed serving system for Transformer-based generative models,' in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[37] W. Kwon, Z. Li, S. Zhuang et al., 'Efficient memory management for large language model serving with PagedAttention,' in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2023, pp. 611–626. DOI: 10.1145/3600006.3613165

[38] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura and R. Bianchini, 'Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,' in *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2017, pp. 153–167. DOI: 10.1145/3132747.3132772

[39] V. N. Padmanabhan and J. C. Mogul, 'Using predictive prefetching to improve World Wide Web latency,' *ACM SIGCOMM Computer Communication Review*, vol. 26, no. 3, pp. 22–36, 1996. DOI: 10.1145/235160.235164

[40] J. Dean and L. A. Barroso, 'The tail at scale,' *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794

[41] D. J. Bernstein, 'Curve25519: New Diffie-Hellman speed records,' in *Public Key Cryptography – PKC 2006*, ser. Lecture Notes in Computer Science, vol. 3958, Springer, 2006, pp. 207–228. DOI: 10.1007/11745853_14

[42] A. Langley, M. Hamburg and S. Turner, 'Elliptic curves for security,' Internet Research Task Force, RFC 7748, Jan. 2016. DOI: 10.17487/RFC7748

[43] H. Krawczyk and P. Eronen, 'HMAC-based extract-and-expand key derivation function (HKDF),' Internet Engineering Task Force, RFC 5869, May 2010. DOI: 10.17487/RFC5869

[44] M. Dworkin, 'Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,' National Institute of Standards and Technology, NIST Special Publication 800-38D, 2007. DOI: 10.6028/NIST.SP.800-38D

[45] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, 'High-speed high-security signatures,' *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012. DOI: 10.1007/s13389-012-0027-1

[46] S. Josefsson and I. Liusvaara, 'Edwards-curve digital signature algorithm (EdDSA),' Internet Research Task Force, RFC 8032, Jan. 2017. DOI: 10.17487/RFC8032

[47] Python Cryptographic Authority, *Pyca/cryptography*, version 50.0.2. Accessed: 2026-10-16. [Online]. Available: https://cryptography.io/

[48] S. Ramírez, *FastAPI*, version 0.143.2. Accessed: 2026-10-16. [Online]. Available: https://fastapi.tiangolo.com/

[49] F. Pedregosa, G. Varoquaux, A. Gramfort et al., 'Scikit-learn: Machine learning in Python,' *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[50] Team SimPy, *SimPy: Discrete event simulation for Python*, version 4.1.2. Accessed: 2026-10-16. [Online]. Available: https://simpy.readthedocs.io/

[51] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor et al., 'Array programming with NumPy,' *Nature*, vol. 585, no. 7825, pp. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2

A Example code

All four programs run with Python 3.11 or later after `pip install cryptography fastapi httpx numpy scikit-learn simpy` (versions tested: `cryptography` 50.0.2, FastAPI 0.143.2, NumPy 2.4.6 [51], scikit-learn 1.9.1, SimPy 4.1.2). The listings below are read from the files in `paper/code/`, and each output block is the saved output of `run_all.py`, so the paper always matches the code. They are teaching implementations: keys live in memory, telemetry is synthetic, and production concerns such as key rotation, persistence and observability are left out.

A.1 `attested_rpc.py`: attested, encrypted invocation (Algorithm 1)

```python
"""attested_rpc.py: attested, encrypted invocation of a cloud-only feature (Algorithm 1).

Implements lines 1-8 and 10-13 of Algorithm 1; the rate limiter (line 9) and the output
watermark (line 13) are left out. The client derives a per-request AES-GCM key from an
ephemeral X25519 exchange with the server and signs (ephemeral key || nonce || timestamp)
with an Ed25519 device key. In production that key would be bound to a platform
attestation verdict (Play Integrity / App Attest); here a locally registered key stands in.

Run:  python attested_rpc.py
"""
import base64, json, os, time
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey, Ed25519PublicKey
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey, X25519PublicKey
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

b64 = lambda b: base64.b64encode(b).decode()
unb64 = base64.b64decode
raw = dict(encoding=serialization.Encoding.Raw, format=serialization.PublicFormat.Raw)

def derive_key(shared, nonce):
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=nonce, info=b"sa-heacs/v1").derive(shared)

# ---- server ----------------------------------------------------------------
SERVER_KEY = X25519PrivateKey.generate()
TRUSTED_DEVICE_KEYS = {}   # stands in for a Play Integrity / App Attest verifier
SEEN_NONCES, MAX_SKEW_S = set(), 30

def proprietary_filter(pixels):
    """The protected IP. It exists only in the server process."""
    mean = sum(pixels) / len(pixels)
    return [round(0.8 * p + 0.2 * mean, 4) for p in pixels]

FEATURES = {"neural_filter_v3": proprietary_filter}

class Envelope(BaseModel):
    device_id: str; client_pub: str; nonce: str; ts: int; signature: str; ciphertext: str

app = FastAPI(title="SA-HEACS secure cluster")

@app.post("/v1/invoke")
def invoke(env: Envelope):
    nonce = unb64(env.nonce)
```

```
    if abs(time.time() - env.ts) > MAX_SKEW_S or nonce in SEEN_NONCES:
        raise HTTPException(401, "stale or replayed request")
    device = TRUSTED_DEVICE_KEYS.get(env.device_id)
    if device is None:
        raise HTTPException(401, "device not attested")
    signed = unb64(env.client_pub) + nonce + str(env.ts).encode()
    try:
        device.verify(unb64(env.signature), signed)
    except Exception:
        raise HTTPException(401, "bad attestation signature")
    SEEN_NONCES.add(nonce)
    key = derive_key(SERVER_KEY.exchange(X25519PublicKey.from_public_bytes(unb64(env.client_pub))),
     ↪ nonce)
    aes = AESGCM(key)
    req = json.loads(aes.decrypt(nonce[:12], unb64(env.ciphertext), signed))
    fn = FEATURES.get(req["feature"])
    if fn is None:
        raise HTTPException(404, "unknown feature")
    out = json.dumps({"result": fn(req["args"])}).encode()
    return {"ciphertext": b64(aes.encrypt(nonce[12:] + b"\x00" * 8, out, signed))}

# ---- client shell: no proprietary logic, only identity and transport --------
class ClientShell:
    def __init__(self, http, server_pub, device_id, device_key):
        self.http, self.server_pub, self.device_id, self.device_key = http, server_pub, device_id,
     ↪ device_key

    def call(self, feature, args):
        eph = X25519PrivateKey.generate()
        nonce, ts = os.urandom(16), int(time.time())
        pub = eph.public_key().public_bytes(**raw)
        signed = pub + nonce + str(ts).encode()
        aes = AESGCM(derive_key(eph.exchange(self.server_pub), nonce))
        body = json.dumps({"feature": feature, "args": args}).encode()
        env = Envelope(device_id=self.device_id, client_pub=b64(pub), nonce=b64(nonce), ts=ts,
                       signature=b64(self.device_key.sign(signed)),
                       ciphertext=b64(aes.encrypt(nonce[:12], body, signed)))
        r = self.http.post("/v1/invoke", json=env.model_dump())
        r.raise_for_status()
        out = aes.decrypt(nonce[12:] + b"\x00" * 8, unb64(r.json()["ciphertext"]), signed)
        return json.loads(out)["result"]

if __name__ == "__main__":
    from fastapi.testclient import TestClient
    device_key = Ed25519PrivateKey.generate()
    TRUSTED_DEVICE_KEYS["device-42"] = device_key.public_key()   # result of a passed attestation
    shell = ClientShell(TestClient(app), SERVER_KEY.public_key(), "device-42", device_key)
    print("server result:", shell.call("neural_filter_v3", [0.1, 0.5, 0.9]))
    rogue = ClientShell(TestClient(app), SERVER_KEY.public_key(), "device-42",
     ↪ Ed25519PrivateKey.generate())
    try:
        rogue.call("neural_filter_v3", [0.1])
    except Exception as e:
        print("unattested clone rejected:", e.response.status_code, e.response.json()["detail"])
```

Output:

```
server result: [0.18, 0.5, 0.82]
unattested clone rejected: 401 bad attestation signature
```

A.2 `orchestrator.py`: routing, micro-batching, prefetch (Algorithms 2–4)

```python
"""orchestrator.py: locality routing, micro-batching and Markov prefetch (Algorithms 2-4).

The security tier is checked in code before any learned component runs, so no model
output can move a cloud_only feature to the client.

Run:  python orchestrator.py
"""
import asyncio, time
from collections import Counter, defaultdict
from dataclasses import dataclass
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier

@dataclass
class Request:
    feature: str
    tier: str            # "cloud_only" | "hybrid" | "client_ok" (set by the lifecycle pipeline)
    payload_kb: float
    est_flops_g: float
    rtt_ms: float
    device_gflops: float
    battery: float

def features(r):
    return [r.payload_kb, r.est_flops_g, r.rtt_ms, r.device_gflops, r.battery]

class LocalityRouter:
    """Security tier is a hard constraint; the model only optimizes inside it."""
    def __init__(self):
        self.model = GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=0)
    def fit(self, X, y):
        self.model.fit(X, y); return self
    def route(self, r):
        if r.tier == "cloud_only":
            return "cloud"
        if r.tier == "client_ok" and r.rtt_ms > 250:
            return "local"
        return "cloud" if self.model.predict_proba([features(r)])[0, 1] >= 0.5 else "local"

def synthetic_telemetry(n=4000, seed=0):
    rng = np.random.default_rng(seed)
    payload = rng.lognormal(4, 1, n); flops = rng.lognormal(2, 1.2, n)
    rtt = rng.gamma(2, 30, n); dev = rng.uniform(5, 200, n); batt = rng.uniform(0, 1, n)
    local_ms = flops / dev * 1000
    remote_ms = rtt + payload * 8 / 50_000 * 1000 + flops / 2000 * 1000
    return np.column_stack([payload, flops, rtt, dev, batt]), (remote_ms < local_ms).astype(int)

class MicroBatcher:
    def __init__(self, batch_fn, window_ms=8.0, max_batch=32):
        self.batch_fn, self.window, self.max_batch = batch_fn, window_ms / 1000, max_batch
        self.queues, self.timers, self.batches_run = defaultdict(list), {}, 0
    async def submit(self, feature, x):
        fut = asyncio.get_running_loop().create_future()
        q = self.queues[feature]; q.append((x, fut))
        if len(q) >= self.max_batch:
            self._flush(feature)
        elif feature not in self.timers:
```

```
            self.timers[feature] = asyncio.get_running_loop().call_later(self.window, self._flush,
    ↪ feature)
        return await fut
    def _flush(self, feature):
        if (t := self.timers.pop(feature, None)) is not None:
            t.cancel()
        items, self.queues[feature] = self.queues[feature], []
        if not items:
            return
        out = self.batch_fn(feature, np.stack([x for x, _ in items]))   # one kernel launch
        self.batches_run += 1
        for (_, fut), y in zip(items, out):
            fut.set_result(y)

class MarkovPrefetcher:
    def __init__(self, k=2, min_p=0.25):
        self.k, self.min_p, self.counts = k, min_p, defaultdict(Counter)
    def observe(self, prev, nxt):
        self.counts[prev][nxt] += 1
    def predict(self, current):
        c = self.counts[current]; total = sum(c.values()) or 1
        return [f for f, n in c.most_common(self.k) if n / total >= self.min_p]

if __name__ == "__main__":
    X, y = synthetic_telemetry()
    router = LocalityRouter().fit(X[:3000], y[:3000])
    print(f"router held-out accuracy: {router.model.score(X[3000:], y[3000:]):.3f}")
    print("cloud_only feature routes to:",
          router.route(Request("neural_filter_v3", "cloud_only", 20, 0.1, 400, 200, 1.0)))
    W = np.random.default_rng(1).normal(size=(16, 16))            # stand-in for protected weights
    batcher = MicroBatcher(lambda f, xb: np.tanh(xb @ W), window_ms=5, max_batch=64)
    async def burst():
        t = time.perf_counter()
        xs = np.random.default_rng(2).normal(size=(200, 16))
        await asyncio.gather(*(batcher.submit("neural_filter_v3", x) for x in xs))
        return time.perf_counter() - t
    secs = asyncio.run(burst())
    print(f"200 requests served in {batcher.batches_run} batched calls ({secs*1000:.1f} ms)")
    pf = MarkovPrefetcher()
    session = ["open", "denoise", "upscale", "export", "open", "denoise", "upscale", "open", "denoise",
    ↪ "colorize"]
    for a, b in zip(session, session[1:]):
        pf.observe(a, b)
    print("after 'denoise', prefetch:", pf.predict("denoise"))
```

Output (the wall-clock time on the third line varies between machines and runs):

```
router held-out accuracy: 0.956
cloud_only feature routes to: cloud
200 requests served in 4 batched calls (40.9 ms)
after 'denoise', prefetch: ['upscale', 'colorize']
```

A.3 `lifecycle.py`: promotion and demotion (Algorithm 5)

```
"""lifecycle.py: Dynamic Feature Promotion/Demotion Pipeline (Algorithm 5).

Simulates one year of signals for a feature whose novelty and revenue share decay while
competitor parity rises and the method is published on day 201.

Run:  python lifecycle.py
```

```
"""
from dataclasses import dataclass, field
from enum import Enum

class Tier(Enum):
    CLOUD_ONLY = "cloud_only"; HYBRID = "hybrid"; CLIENT_OK = "client_ok"

@dataclass
class FeatureSignals:
    novelty: float; revenue_share: float; competitor_parity: float; published: bool
    cost_per_1k_usd: float; p95_latency_ms: float; on_device_feasible: bool

WEIGHTS = dict(novelty=0.35, revenue=0.35, parity=0.20, published=0.10)

def ip_value(s):
    return round(WEIGHTS["novelty"] * s.novelty + WEIGHTS["revenue"] * s.revenue_share
                 + WEIGHTS["parity"] * (1 - s.competitor_parity)
                 + WEIGHTS["published"] * (0 if s.published else 1), 3)

def serving_pressure(s, cost_budget=2.0, latency_budget=120.0):
    return round(min(1.0, 0.5 * s.cost_per_1k_usd / cost_budget + 0.5 * s.p95_latency_ms /
     ↪ latency_budget), 3)

@dataclass
class Feature:
    name: str
    tier: Tier = Tier.CLOUD_ONLY
    days_in_tier: int = 0
    history: list = field(default_factory=list)

PROTECT_HI, PROTECT_LO, MIN_DWELL_DAYS = 0.65, 0.40, 30

def decide(f, s):
    v, p = ip_value(s), serving_pressure(s)
    if v >= PROTECT_HI and f.tier != Tier.CLOUD_ONLY:
        return Tier.CLOUD_ONLY, f"demote: ip_value {v} >= {PROTECT_HI}"
    if f.days_in_tier < MIN_DWELL_DAYS:
        return f.tier, "hold: dwell time"
    if f.tier == Tier.CLOUD_ONLY and v < PROTECT_HI and p > 0.6:
        return Tier.HYBRID, f"promote: ip_value {v}, pressure {p}"
    if f.tier == Tier.HYBRID and v < PROTECT_LO and s.on_device_feasible:
        return Tier.CLIENT_OK, f"promote (needs approval): ip_value {v} < {PROTECT_LO}"
    return f.tier, "hold"

def step(f, s, approve):
    target, why = decide(f, s)
    if target == Tier.CLIENT_OK and f.tier != Tier.CLIENT_OK and not approve(f, why):
        target, why = f.tier, why + " -> rejected by reviewer"
    if target != f.tier:
        f.history.append((f.tier.value, target.value, why)); f.tier, f.days_in_tier = target, 0
    else:
        f.days_in_tier += 1
    return f

if __name__ == "__main__":
    f = Feature("neural_filter_v3")
    for day in range(365):
        t = day / 365
        s = FeatureSignals(novelty=0.95 - 0.8 * t, revenue_share=0.6 - 0.45 * t,
```

```
                            competitor_parity=min(1, 0.1 + 1.1 * t), published=day > 200,
                            cost_per_1k_usd=1.8, p95_latency_ms=110, on_device_feasible=day > 150)
        before = len(f.history)
        step(f, s, approve=lambda feat, why: True)
        if len(f.history) > before:
            old, new, why = f.history[-1]
            print(f"day {day:>3}: {old:>10} -> {new:<10} {why}")
    print("final tier:", f.tier.value)
```

Output:

```
day  97: cloud_only -> hybrid     promote: ip_value 0.648, pressure 0.908
day 201:     hybrid -> client_ok  promote (needs approval): ip_value 0.36 < 0.4
final tier: client_ok
```

A.4 `simulate.py`: Section 8 simulation

The batching experiment is a SimPy process model: an arrival process appends to a per-feature queue, a window timer or the 64-item cap flushes it, and each batch waits for a GPU from a `simpy.Resource` pool. The prefetch experiment generates Markov sessions and scores a top-k predictor fitted on the first third. The program writes `results_batching.csv` and `results_prefetch.csv`, which Figure 3, Table 3 and Figure 4 read directly.

```
"""simulate.py: discrete-event cost/latency model behind Section 8 (Figures 3 and 4).

Experiment 1 (batching window, SimPy)
    One protected feature. Requests arrive as a Poisson process (2,000 req/s). The
    orchestrator holds requests for the feature until the window W expires (timer starts
    at the first queued request) or 64 requests are queued, then sends one batched kernel
    call to a pool of GPUs. A call on b items costs 4.0 ms + 0.15 ms * b. The pool is sized
    so that expected utilisation is at most 70%. Server latency = arrival -> result, with
    no network round trip. Each configuration simulates 20 s (about 40,000 requests).

Experiment 2 (prefetch depth k)
    Sessions are walks on a skewed first-order Markov chain over 12 features. A first-order
    model is fitted on the first third of the sessions; on the rest, after each step the
    orchestrator speculatively runs the k most likely next features. A hit is served from
    the client cache (5 ms); a miss pays the network round trip (40 ms) plus one unbatched
    kernel call (4.15 ms). Extra compute = speculative runs that were never used, as a
    percentage of demand runs.

All costs are assumptions, not measurements. Output: results_batching.csv,
results_prefetch.csv (next to this file) and a summary on stdout.

Run:  python simulate.py
"""
import csv, math
from pathlib import Path
import numpy as np
import simpy

HERE = Path(__file__).resolve().parent

# ---- assumptions (Section 8.1) ------------------------------------------------
RATE = 2000.0          # requests per second
FIXED_MS = 4.0         # per kernel call
PER_ITEM_MS = 0.15     # per item in a batch
MAX_BATCH = 64
TARGET_UTIL = 0.70
```

```
SIM_S = 20.0
WARMUP_S = 1.0         # discard the first second
WINDOWS_MS = [0, 1, 2, 5, 10, 20, 32]
RTT_MS, LOCAL_HIT_MS = 40.0, 5.0
N_FEATURES, N_SESSIONS, SESSION_LEN, KS = 12, 3000, 20, [0, 1, 2, 3, 4]


def kernel_ms(b):
    return FIXED_MS + PER_ITEM_MS * b


def pool_size(window_ms):
    """GPUs needed for <= 70% utilisation, from the expected batch size."""
    b = min(MAX_BATCH, 1 + RATE * window_ms / 1000)
    busy = RATE / b * kernel_ms(b) / 1000          # GPUs busy on average
    return max(1, math.ceil(busy / TARGET_UTIL))


def run_batching(window_ms, seed=0):
    rng = np.random.default_rng(seed)
    env = simpy.Environment()                          # time unit: ms
    gpus = simpy.Resource(env, capacity=pool_size(window_ms))
    queue, timer, latencies, gpu_ms, batch_sizes = [], [None], [], [0.0], []

    def execute(items):
        with gpus.request() as req:
            yield req
            cost = kernel_ms(len(items))
            yield env.timeout(cost)
        if items[0] >= WARMUP_S * 1000:
            gpu_ms[0] += cost
            batch_sizes.append(len(items))
        latencies.extend(env.now - t for t in items if t >= WARMUP_S * 1000)

    def flush():
        if timer[0] is not None and timer[0].is_alive and env.active_process is not timer[0]:
            timer[0].interrupt()
        timer[0] = None
        items = queue[:]
        queue.clear()
        if items:
            env.process(execute(items))

    def window_timer():
        try:
            yield env.timeout(window_ms)
            timer[0] = None
            flush()
        except simpy.Interrupt:
            pass

    def arrivals():
        while env.now < SIM_S * 1000:
            yield env.timeout(rng.exponential(1000 / RATE))
            queue.append(env.now)
            if window_ms == 0 or len(queue) >= MAX_BATCH:
                flush()
            elif timer[0] is None:
                timer[0] = env.process(window_timer())
```

```
    env.process(arrivals())
    env.run()
    lat = np.array(latencies)
    return dict(window_ms=window_ms, gpus=gpus.capacity, requests=len(lat),
                mean_batch=round(float(np.mean(batch_sizes)), 2),
                gpu_ms_per_1k=round(gpu_ms[0] / len(lat) * 1000, 1),
                utilisation=round(gpu_ms[0] / (gpus.capacity * (SIM_S - WARMUP_S) * 1000), 3),
                p50_ms=round(float(np.percentile(lat, 50)), 1),
                p95_ms=round(float(np.percentile(lat, 95)), 1),
                p99_ms=round(float(np.percentile(lat, 99)), 1))


def markov_chain(rng, decay=0.55):
    """Each row: geometric preferences r^rank over a random ordering, lightly perturbed."""
    P = np.zeros((N_FEATURES, N_FEATURES))
    for a in range(N_FEATURES):
        w = decay ** np.arange(N_FEATURES) * rng.uniform(0.8, 1.2, N_FEATURES)
        P[a, rng.permutation(N_FEATURES)] = w
    return P / P.sum(axis=1, keepdims=True)


def run_prefetch(seed=1):
    rng = np.random.default_rng(seed)
    P = markov_chain(rng)
    sessions = []
    for _ in range(N_SESSIONS):
        s = [rng.integers(N_FEATURES)]
        for _ in range(SESSION_LEN - 1):
            s.append(rng.choice(N_FEATURES, p=P[s[-1]]))
        sessions.append(s)
    train, test = sessions[:N_SESSIONS // 3], sessions[N_SESSIONS // 3:]
    counts = np.zeros((N_FEATURES, N_FEATURES))
    for s in train:
        for a, b in zip(s, s[1:]):
            counts[a, b] += 1
    ranking = np.argsort(-counts, axis=1, kind="stable")
    miss_ms = RTT_MS + kernel_ms(1)
    rows = []
    for k in KS:
        hits = steps = 0
        for s in test:
            for a, b in zip(s, s[1:]):
                steps += 1
                hits += b in ranking[a, :k]
        h = hits / steps
        rows.append(dict(k=k, hit_rate=round(h, 3),
                         mean_latency_ms=round(h * LOCAL_HIT_MS + (1 - h) * miss_ms, 1),
                         extra_compute_pct=round((k * steps - hits) / steps * 100, 1)))
    return rows


def write_csv(path, rows):
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader(); w.writerows(rows)


if __name__ == "__main__":
```

```
    batching = [run_batching(w) for w in WINDOWS_MS]
    write_csv(HERE / "results_batching.csv", batching)
    print(f"{'W ms':>5} {'GPUs':>4} {'batch':>6} {'GPU-ms/1k':>9} {'util':>5} {'p50':>5} {'p95':>5}
     ↪ {'p99':>5}")
    for r in batching:
        print(f"{r['window_ms']:>5} {r['gpus']:>4} {r['mean_batch']:>6} {r['gpu_ms_per_1k']:>9} "
              f"{r['utilisation']:>5} {r['p50_ms']:>5} {r['p95_ms']:>5} {r['p99_ms']:>5}")
    prefetch = run_prefetch()
    write_csv(HERE / "results_prefetch.csv", prefetch)
    print(f"\n{'k':>2} {'hits':>5} {'mean ms':>7} {'extra %':>7}")
    for r in prefetch:
        print(f"{r['k']:>2} {r['hit_rate']:>5} {r['mean_latency_ms']:>7} {r['extra_compute_pct']:>7}")
```

Output:

```
W ms GPUs  batch GPU-ms/1k  util   p50   p95   p99
   0   12    1.0    4150.0 0.693   4.1   5.0   6.0
   1    5    3.0    1484.6 0.595   5.3   5.8   6.0
   2    3    5.0     950.3 0.635   6.1   7.1   7.4
   5    2  11.02     513.0 0.514   8.6  10.9  11.4
  10    1  21.02     340.3 0.682  12.6  17.2  18.0
  20    1   41.1     247.3 0.495  20.5  29.8  31.0
  32    1  61.32     215.0 0.431  28.3  42.7  44.7

k  hits mean ms extra %
0   0.0    44.1     0.0
1  0.44    26.9    56.0
2 0.682    17.4   131.8
3 0.827    11.8   217.3
4 0.906     8.7   309.4
```